

校园信息聚合论坛系统（校园通）组内工作量说明文档

一、项目基本信息

- 项目名称：校园信息聚合论坛系统（校园通）

二、个人参与的工作内容

1. 开发工作

1.1 后端开发

技术栈：Python Django + Django REST Framework + MySQL

主要工作：

- 项目框架搭建：
 - 将后端技术栈从Express.js重构为Python Django框架
 - 设计并实现了模块化的Django应用结构（accounts、content、likes、reports、ads、favorites、follows、browsing_history、ai_assistant等）
 - 配置了Django REST Framework用于API开发
 - 设置了数据库连接，使用密码3147
- 用户系统：
 - 实现了自定义User模型和JWT认证系统
 - 扩展了User模型，添加了隐私设置字段（show_profile、allow_messages、allow_follow、show_posts、save_history）
 - 实现了用户注册、登录、获取当前用户信息等功能
- 内容管理系统：
 - 实现了帖子的CRUD功能，支持多种帖子类型
 - 实现了评论系统，支持评论的发布、点赞、删除等功能
 - 实现了板块管理功能，支持板块的创建、更新、删除和查询
 - 修改了Post模型，将images字段改为media_files，支持图片和视频URL数组
- 社交功能：
 - 实现了点赞功能，支持对帖子和评论的点赞
 - 实现了收藏功能，支持收藏/取消收藏帖子
 - 实现了关注功能，支持关注/取消关注用户
 - 实现了私信功能，支持文字、图片、视频等多种消息类型
 - 实现了浏览历史功能，记录用户浏览帖子的历史
- 管理功能：
 - 实现了管理员权限系统，支持不同角色的权限管理
 - 实现了举报功能，支持用户举报帖子/评论，管理员处理举报

- 实现了广告管理功能，支持广告的发布、审核、删除等功能

- **AI功能集成：**

- 创建了ai_assistant应用，实现了校园知识问答机器人的核心功能
- 集成了DeepSeek API，实现智能问答、帖子创作辅助和智能搜索增强等功能
- 实现了知识库管理系统，支持常见问题和答案的存储和管理
- 实现了AI内容审核功能，增强了敏感词过滤机制

1.2 前端开发

技术栈： React + Vite + Ant Design + Axios

主要工作：

- **项目框架搭建：**

- 搭建了前端基础框架（React + Vite + Ant Design）
- 配置了前后端通信（API代理、跨域处理）
- 创建了前端项目的package.json文件，添加了必要依赖

- **页面开发：**

- 实现了登录、注册页面
- 实现了首页，包含帖子列表、搜索功能、AI助手等
- 实现了帖子详情页，支持评论、点赞、收藏、关注等功能
- 实现了发帖页面，支持富文本编辑器、图片/视频上传等
- 实现了个人中心页面，包含用户信息、发帖历史、收藏夹、关注列表等
- 实现了管理后台，包含用户管理、内容管理、板块管理、广告管理、举报管理等
- 实现了设置中心页面，包含账号管理、密码管理、隐私管理、通知设置等

- **组件开发：**

- 开发了PostList组件，实现帖子列表展示、关键词搜索、板块筛选等功能
- 开发了PostDetail组件，实现帖子详情展示、评论、点赞等功能
- 开发了PostForm组件，实现发帖功能，支持AI辅助功能
- 开发了CommentList组件，实现评论列表展示、点赞、回复等功能
- 开发了AIAssistant组件，实现校园知识问答机器人界面
- 开发了各种管理组件，如UserManagement、ContentManagement等

- **状态管理与路由：**

- 使用Context API实现了登录状态管理（AuthContext）
- 实现了前端路由配置，添加了完整的页面路由
- 实现了权限控制，只有管理员才能看到管理后台入口

1.3 数据库设计与优化

- 设计了规范化的数据库模型，减少数据冗余
- 实现了数据库索引优化，为经常查询的字段创建索引
- 开发了数据库迁移脚本，管理数据库结构的变化

- 实现了数据的软删除和恢复功能
- 在后端数据库中创建了5个预定义板块：校园公告、二手交易、学习交流、休闲娱乐、广告专区

2. 测试工作

- **API测试**：使用PowerShell测试API端点，确认API相关端点返回200 OK响应
- **功能测试**：测试了所有功能，确保能正常工作，包括用户注册、登录、发帖、评论、点赞、收藏、关注、私信等
- **性能测试**：进行了性能测试，优化了系统性能
- **兼容性测试**：确保系统在不同浏览器和设备上能正常运行

3. 排错工作

- **前端错误修复**：
 - 修复了图标导入错误，如将不存在的BriefcaseOutlined替换为FileTextOutlined
 - 修复了useNavigate的错误使用方式，将`const [navigate] = useNavigate()`改为`const navigate = useNavigate()`
 - 修复了Spin组件的tip属性使用问题，改为使用自定义布局
 - 修复了组件弃用属性，如将bodyStyle替换为styles={{ body: { padding: '16px 24px 20px' } }}
- **后端错误修复**：
 - 修复了likes应用views.py中的语法错误
 - 修复了reports/views.py中的Response调用语法错误
 - 修复了reports/serializers.py中的未终止字符串
 - 修复了content/urls.py中的括号问题
 - 修复了reports/apps.py中的字符串问题
- **API错误修复**：
 - 修复了API URL配置，确保所有请求都使用带斜杠的URL
 - 解决了Django APPEND_SLASH机制导致的POST请求500错误
 - 修复了广告管理API 500错误，添加了缺失的return value语句
 - 修复了板块API 500错误，为板块详情URL添加了尾部斜杠
 - 修复了管理员评论API路由，创建了专门的管理员评论路由配置
- **功能错误修复**：
 - 修复了评论点赞数量增加两次的问题，移除了冗余的点赞数增减逻辑
 - 修复了帖子列表和详情页显示"Invalid Date"的问题
 - 修复了个人中心点击无响应的问题，检查并修复了前端路由配置
 - 修复了发布新帖按钮无法点击的问题，使用useNavigate钩子实现程式化导航
 - 修复了私信发送时返回500错误的问题，解决了MessageCreateSerializer中的recipient_id字段配置错误
 - 修复了视频上传失败的问题，移除了axios实例中显式设置的Content-Type

4. 文档书写

- 编写了校园信息聚合论坛系统（校园通）的项目设计文档v1.0和v1.4

- 编写了API设计文档v1.0和v1.4
- 编写了数据库设计文档v1.0和v1.4
- 生成了数据库初始化脚本
- 编写了《数据库创建与初始化指南》，详细说明了数据库的创建和初始化步骤
- 编写了《启动项目指南》，包含数据库初始化、后端启动和前端启动的完整流程
- 编写了部署文档，详细说明了部署步骤和注意事项
- 编写了个人开发日志，记录了每天的开发成果、问题分析与解决、技术要点等

5. 开发环境设置/测试

- 搭建了后端开发环境，安装了必要的依赖（Django、Django REST Framework、MySQL等）
- 搭建了前端开发环境，安装了必要的依赖（React、Ant Design、Axios等）
- 配置了前后端项目的启动脚本
- 解决了启动项目时遇到的npm start命令错误问题
- 运行了数据库迁移命令，创建了所有必要的数据库表
- 启动了后端开发服务器，验证了API端点的正确性
- 启动了前端开发服务器，验证了页面的正确性和功能的完整性
- 运行了前端构建命令，确保代码没有语法错误

三、工作成果总结

通过这段时间的开发，我成功实现了校园信息聚合论坛系统（校园通）的核心功能，包括：

1. 完整的用户系统，支持注册、登录、权限管理等功能
2. 完善的内容管理系统，支持帖子、评论、板块的管理
3. 丰富的社交功能，包括点赞、收藏、关注、私信等
4. 强大的管理功能，支持用户管理、内容管理、广告管理、举报管理等
5. 智能的AI功能，包括智能问答、帖子创作辅助、智能搜索增强等
6. 美观的前端界面，支持响应式设计，适配不同屏幕尺寸
7. 稳定的后端服务，支持高并发访问

系统采用了前后端分离架构，技术栈先进，代码结构清晰，具有良好的可扩展性和可维护性。在开发过程中，我遇到了各种技术挑战，但通过分析错误日志、调试代码和查阅文档，成功解决了这些问题，确保了系统的稳定性和可靠性。

四、后续工作计划

1. 优化系统性能，提高页面加载速度和响应性
2. 添加更多测试用例，确保系统功能的正确性
3. 完善系统的错误处理和异常管理
4. 优化用户体验，添加更多交互效果
5. 扩展AI功能，提升AI生成内容的质量和相关性
6. 实现更多的社交功能，如话题讨论、兴趣小组等
7. 优化移动端适配，提升移动端用户体验
8. 部署系统到生产环境，确保系统能稳定运行

五、技术难点与解决方案

1. 前后端数据格式不一致

- **问题**：前端使用camelCase，后端使用snake_case
- **解决方案**：在前端API调用中添加数据转换逻辑，将后端返回的snake_case转换为camelCase

2. JWT token过期处理

- **问题**：token过期后需要重新登录
- **解决方案**：实现axios响应拦截器，当遇到401错误时自动刷新token

3. AI功能集成

- **问题**：sklearn库依赖问题，API调用失败处理
- **解决方案**：实现sklearn优雅降级方案，当sklearn库不可用时自动切换到基于Jaccard相似度的简单实现；添加备用方案，以防API调用失败

4. 多媒体支持

- **问题**：视频上传和展示
- **解决方案**：在Post模型中添加media_files字段，支持图片和视频URL数组；根据文件MIME类型判断文件类型，而不是URL扩展名；在axios请求拦截器中添加FormData类型检测，移除默认的Content-Type

5. 私信功能实现

- **问题**：私信发送时返回500错误，私信图片加载问题
- **解决方案**：修复了MessageCreateSerializer中的recipient_id字段配置错误；修正了前端消息详情页面的图片URL生成逻辑，将API URL转换为媒体文件直接访问URL

6. 性能优化

- **问题**：页面加载速度慢
- **解决方案**：优化PostList组件的性能，减少不必要的渲染；实现组件的按需加载，提高初始页面加载速度；为频繁查询的字段添加索引，提高数据库查询性能